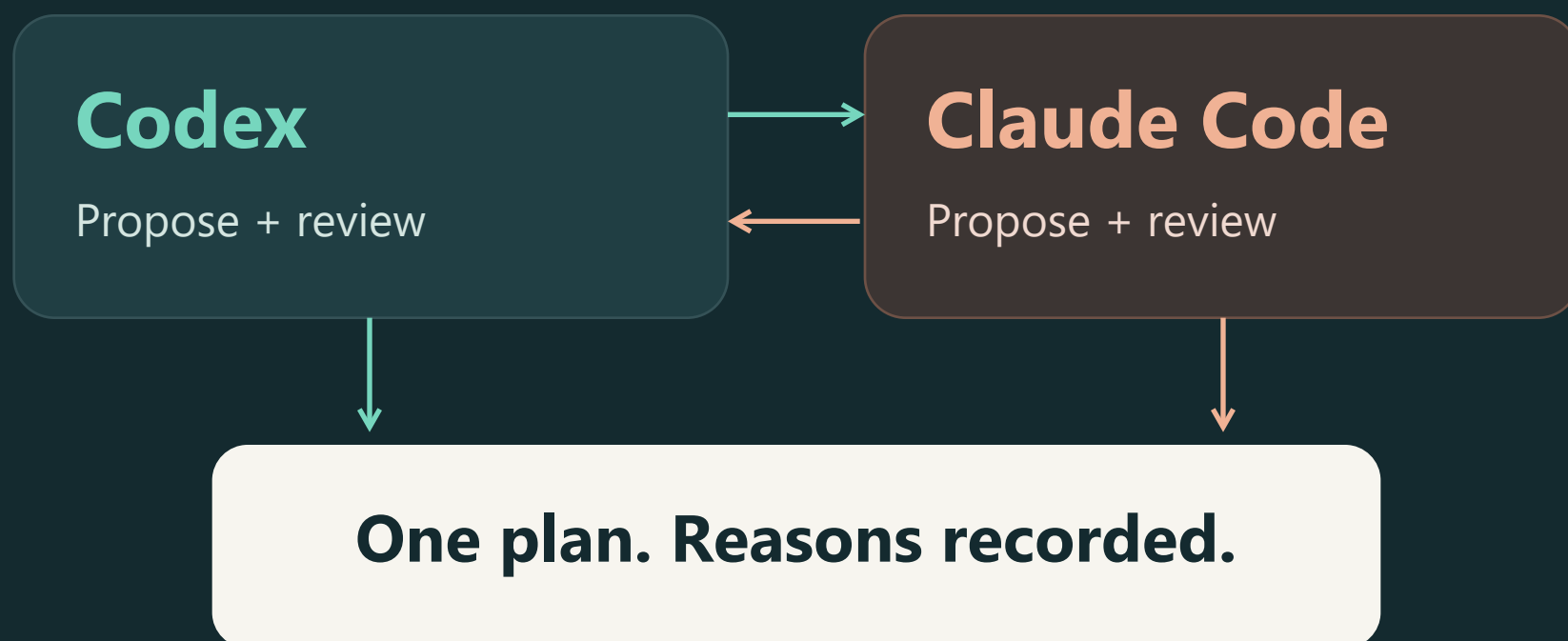


Before you code, let two AIs challenge the plan.

Codex + Claude Code.

Two perspectives. One shared plan.



A shared skill for small features
and whole-project planning.

Your chat leads. The other AI reviews.

Start in either app. C2C calls the other provider through its signed-in command-line tool.

01 Understand the project

Check what exists, production evidence and the goal.

02 Compare approaches

Create separate proposals in independent plan mode.

03 Challenge the details

Review trade-offs, security risks and test coverage.

04 Record the decision

Revise the plan, verify it and keep open issues visible.

SECURITY + TEST PLAN

You review the plan before coding.

Clear direction. Checks that matter.

01 **Project preflight**

Inspect evidence. Separate production use from readiness. Keep unknowns visible.

02 **Clarify before committing**

Resolve important goals and constraints; name the first build step.

03 **Security + acceptance checks**

Review risks proportionally and specify meaningful tests. Implementation still needs testing.

04 **Task sizing + model advice**

Recommend suitable effort and models. Change no model settings automatically.

05 **Visible, durable decisions**

Record proposals, challenges and accepted, rejected or unresolved findings in Markdown.

[135 C2C runtime tests passed](#) | Windows + Ubuntu | Node 18, 22, 24

Selected context goes to the peer only after an explicit collaboration request.

Set up the two command-line tools.

For use in both directions, install Node.js 18+ and both CLIs. Follow these official guides.

[Node.js download](#)[Codex CLI setup](#)[Claude Code setup](#)

Then sign in from your terminal

Claude Code

```
claude auth login
```

Codex

```
codex login
```

Using only one direction? Install the other provider's CLI. C2C does not install CLIs or sign you in.

Download once. Install for both apps.

Use PowerShell on Windows or Terminal on macOS/Linux.

1 Download and extract the ZIP

[Download C2C](#)

2 Open a terminal in the extracted folder that contains **scripts**. Run:

```
node scripts/install.mjs
```

3 Check the connection

```
node scripts/council.mjs doctor
```

Installs the skill for your user. No npm install needed.

Check the result. Then open a new chat.

Find the value for the app you will start from:

Starting in Codex

```
"codex_chat_ready": true
```

Codex can call Claude Code.

Starting in Claude Code

```
"claude_chat_ready": true
```

Claude Code can call Codex.

Next: open your project and start a new chat so the app discovers C2C.

False? Follow the reported sign-in or CLI compatibility guidance, then run doctor again.

Setup check only; no planning request is sent.

Start inside Codex.

PASTE INTO YOUR PROJECT CHAT

\$C2C

Use **\$C2C** to plan a car-selling website.

Check the project first and ask important questions. Start with listings, filters, car details, seller contact and an admin dashboard.

Show the disagreements and plan changes. Include security and acceptance tests. Show me the plan before coding.

Codex coordinates.

Claude Code contributes through its CLI.
The current chat manages the exchange for you.

Start inside Claude Code.

PASTE INTO YOUR PROJECT CHAT

/C2C

/C2C Plan a car-selling website with Codex.

Check the project first and ask important questions. Start with listings, filters, car details, seller contact and an admin dashboard.

Show the disagreements and plan changes. Include security and acceptance tests. Show me the plan before coding.

Claude Code coordinates.

Codex contributes through its CLI.

The current chat manages the exchange for you.

Propose. Challenge. Improve the plan.

ILLUSTRATIVE EXCHANGE

CODEX / PROPOSAL

Add customer accounts and saved cars to the MVP.

CLAUDE / CHALLENGE

Can we launch with browsing and seller contact? Keep inventory editing behind admin access.

CODEX / DECISION

Defer customer accounts. Keep listings, filters, seller contact and secured inventory management.

Actual updates arrive after each stage. Open disagreements stay in DISCUSSION.md.

Match the depth to the task.

Small feature

2 PEER CALLS

Focused review + final verification. Check one candidate plan, edge cases and acceptance tests.

Project / open design

3 PEER CALLS

Independent draft + review + verification. Define the MVP, milestones, dependencies and first build step.

Keep the useful outputs

`final-plan.md`

Chosen direction and next steps

`DISCUSSION.md`

Challenges, responses and open issues

`RESULT.md`

Review outcome and remaining gaps

Counts are successful calls. Failed attempts count toward limits.

Give the reviewers a clear starting point.

Use the brief to explain the goal, constraints and evidence. Keep important unknowns visible.

GOAL + BOUNDARIES

Describe the outcome, what is in scope and what must wait. State any firm constraints.

SELECTED EVIDENCE

Share relevant files and observed behavior. Separate confirmed facts from assumptions.

DECISIONS + NEXT STEP

Read the chosen approach, unresolved issues and acceptance checks before implementation.

[Read the setup guide](#) for current commands and troubleshooting. This historical guide describes the v0.7.0 workflow.

Two perspectives. One shared plan.

Start with one feature. Ask both agents to challenge the approach before implementation.

[Open C2C on GitHub](#)

ZiadHassanein/
C2C



[SAVE THIS GUIDE](#) / [SHARE YOUR FIRST REVIEW](#)

Selected context is sent to the other provider; normal account usage applies. Exclude secrets. Security reviews and test plans still need implementation checks.

SETUP & SOURCES

[C2C setup](#)[Codex CLI](#)[Claude Code](#)[Node.js](#)

By Ziad Hassanein | Guide for v0.7.0 | 07 Oct 2026